Parallel and Distributed Computing

Assignment 2

Parallel Triangle Counting

Submitted by

Name: Muhammad Moiz Ansari

Roll No: 23i-0523

Section: F

Semester-6

Submitted to

Sir Aadil Ur Rehman

Department of Computer Science National University of Computer and Emerging Sciences Islamabad, Pakistan

Table of Contents

1. Baseline Algorithm

Triangle counting is a fundamental graph analytics primitive used in community detection, clustering coefficient computation, and network analysis. A triangle is a set of three vertices {vi, vj, vk} such that edges (vi,vj), (vi,vk), and (vj,vk) all exist in the graph.

1.1 Algorithm Design

The baseline implements the list-based approach from Tom et al. [1]. For each vertex vi and its higher-numbered neighbor vj, the algorithm counts common vertices in their adjacency lists — each such common vertex vk forms a triangle (vi, vj, vk). This <vi, vj, vk> ordering ensures each triangle is counted exactly once.

The core intersection uses the scalar sort-merge algorithm from Zhang et al. [2] (Algorithm 2): two pointers traverse sorted adjacency lists simultaneously, advancing the smaller pointer and incrementing the count on a match. Complexity is O(|Adj(vi)| + |Adj(vj)|) per edge.

1.2 Degree-Based Vertex Reordering

Vertices are renumbered in non-decreasing degree order before any counting begins. This is the single most impactful optimization: high-degree vertices receive large IDs, making their upper-triangular adjacency lists very short. Tom et al. [1] report this reduces the maximum working adjacency list size from hundreds to tens of elements on real-world graphs, directly reducing intersection cost.

The upper-triangular CSR (Compressed Sparse Row) representation stores only neighbors with ID greater than the current vertex, enabling cache-friendly linear scans and correct duplicate-free triangle enumeration.

1.3 Why List-Based over Map-Based

Tom et al. [1] also propose hash-map based approaches (hmap-ijk, hmap-jikv1/v2). However, Zhang et al. [2] (Observation 1, Section IV-a) show that hashing only benefits intersections where one list is 10x larger than the other. In real graphs after degree reordering, most intersections have similarly-sized lists (n1 ≈ n2), so hashing provides no advantage. The list-based approach is also directly amenable to SIMD acceleration, which is the primary goal of this assignment.

---

2. SIMD Intersection Method

2.1 Hardware Capability

The target system supports AVX2, providing 256-bit SIMD registers capable of processing 8 x 32-bit integers per operation. This is the highest supported SIMD width on this machine (AVX-512 is not available).

Property	Value

Processor	x86-64 (Intel-compatible)
Logical Threads	8
Max OpenMP Threads	8
SSE 4.2 Support	Yes (128-bit, 4 x int32 lanes)
AVX Support	Yes (256-bit)
AVX2 Support	Yes — 256-bit, 8 x int32 lanes
AVX-512 Support	No
SIMD Register Width	256 bits
Compiler	GCC with -O3 -mavx2 -mpopcnt -fopenmp
OS	Linux (Ubuntu 24)
OpenMP Threads Used	8 (all logical threads)

Table 1: Hardware configuration and SIMD capabilities.

2.2 AVX2 Intrinsic-Based Intersection (V2)

The SIMD intersection (V2) is based on the block-comparison algorithm from Zhang et al. [2] (Figure 1 and 2), adapted from SSE (4 elements) to AVX2 (8 elements). The key steps are:

(1) Load 8 integers from each list into 256-bit YMM registers using _mm256_loadu_si256. (2) Perform all-to-all comparison: rotate list B's block through 8 positions using _mm256_permutevar8x32_epi32 (the AVX2 equivalent of the paper's _mm_shuffle), comparing against list A's block at each rotation using _mm256_cmpeq_epi32. (3) OR all 8 comparison results, convert to bitmask via _mm256_movemask_ps, and count set bits using _mm_popcnt_u32. (4) Advance the pointer whose block has the smaller tail element, as both lists are sorted. A scalar tail handler processes remaining elements when fewer than 8 remain in either list.

Note: Zhang et al. used _mm_shuffle_epi32 for SSE. On AVX2, _mm256_shuffle_epi32 is incorrect because it operates independently within each 128-bit lane and cannot move elements across the lane boundary. _mm256_permutevar8x32_epi32 is the correct replacement as it treats all 8 elements as a single unit.

2.3 OpenMP SIMD Directive (V3)

V3 replaces manual intrinsics with #pragma omp simd, letting the compiler auto-vectorize. The intersection loop is restructured from a pointer-chasing merge (which has data-dependent branching the compiler cannot vectorize) into a fixed-stride inner loop: for each element A[i], scan all of B using the branchless expression temp += (A[i] == B[j]). The #pragma omp simd reduction(+:temp) directive instructs the compiler to vectorize the inner loop.

This changes the complexity from O(sizeA + sizeB) in V2 to O(sizeA x sizeB). For short adjacency lists (common after degree reordering), the SIMD throughput compensates. However, for large lists (D2, D3), this quadratic cost explains V3's sub-1x speedup over the scalar baseline.

3. OpenMP Parallelization Strategy

3.1 Outer Loop Decomposition

Following Tom et al. [1] Section 4.3, parallelization targets the outermost loop (over vertex vi). Each vertex's triangle counting is fully independent — no shared writes occur other than the accumulated count. The OpenMP reduction clause gives each thread a private accumulator, merged atomically at the end, eliminating race conditions.

3.2 Dynamic Scheduling

Real-world graphs exhibit power-law degree distributions: a small fraction of vertices have very high degree while most have low degree. After degree reordering, processing cost per vertex is highly skewed. Static scheduling would leave most threads idle while a few process expensive high-degree vertices.

Dynamic scheduling with chunk size 16 (as used by Tom et al. [1]) solves this: threads grab 16 vertices at a time from a shared queue, naturally balancing load. Chunk size 16 provides fine enough granularity for good load balance while keeping scheduling overhead low.

3.3 Thread Count Justification

The system has 8 logical threads (4 physical cores with 2-way hyperthreading). All 8 threads are used. For compute-bound workloads like triangle counting, hyperthreading contributes approximately 20-30% additional throughput by hiding memory latency. Thread count is set explicitly via omp_set_num_threads(8) and verified at runtime.

---

4. Hybrid Algorithm Design

The hybrid design exploits two independent levels of parallelism simultaneously: task-level parallelism across vertices (OpenMP threads) and data-level parallelism within each intersection (SIMD). Tom et al. [1] Section 4.3 explicitly motivates this combination.

4.1 Variant 1: OMP Threads + AVX2 (V5)

The outer loop is parallelized with #pragma omp parallel for schedule(dynamic, 16) reduction(+:total). Each thread independently calls intersectCountSIMD() for its assigned vertices. The AVX2 rotation index vectors (rotation constants) are defined as const __m256i outside the parallel region to avoid redundant initialization per thread. Since intersectCountSIMD() uses only stack-local variables and read-only constants, it is fully thread-safe with no additional synchronization.

4.2 Variant 2: OMP Threads + OMP SIMD (V6)

Identical outer structure to V5, but each thread calls intersectCountOMPSIMD() which uses #pragma omp simd on the inner loop. This variant is fully portable — it works on any SIMD-capable architecture without

architecture-specific intrinsics. Performance is generally lower than V5 due to the quadratic intersection complexity and less aggressive vectorization compared to manually tuned intrinsics.

4.3 Version Summary

File	Version	Key Technique	Compiler Flags
V1	Scalar Baseline	Sequential sort-merge intersection	-O3
V2	AVX2 SIMD	Manual AVX2 intrinsics, 8-way block compare	-O3 -mavx2 -mpopcnt
V3	OMP SIMD	#pragma omp simd on inner loop	-O3 -mavx2 -fopenmp
V4	OMP Threads	Parallel outer loop, dynamic(16) schedule	-O3 -fopenmp
V5	Hybrid OMP+AVX2	OMP threads + AVX2 intersection kernel	-O3 -mavx2 -mpopcnt -fopenmp
V6	Hybrid OMP+SIMD	OMP threads + #pragma omp simd kernel	-O3 -mavx2 -fopenmp

Table 2: Summary of all six implementations.

---

5. Experimental Setup

5.1 Hardware and Software

All experiments were conducted on the same machine under identical compilation settings. The system details are reported in Table 1. Compilation used GCC with -O3 for all versions, with additional flags per version as listed in Table 2.

5.2 Datasets

Dataset	Vertices	Edges	Triangles	Source
email-Eu-core (D1)	1,005	25,571	105,461	SNAP
ego-Facebook (D2)	4,039	88,234	1,612,010	SNAP
com-LiveJournal (D3)	3,997,962	34,681,189	177,820,130	SNAP

Table 3: Graph datasets used in experiments (all sourced from SNAP).

All datasets were preprocessed identically: self-loops removed, duplicate edges eliminated, vertices reordered by non-decreasing degree, and upper-triangular CSR constructed with sorted adjacency lists.

---

6. Performance Results

6.1 Single-Thread Comparison

Version	D1 Time (ms)	D1 Speedup	D2 Time (ms)	D2 Speedup	D3 Time (ms)	D3 Speedup
V1: Scalar Baseline	2.195	1.00x	12.321	1.00x	7477.61	1.00x
V2: AVX2 SIMD	0.896	2.45x	4.972	2.48x	5235.09	1.43x
V3: OMP SIMD	1.989	1.10x	21.463	0.57x	11807.0	0.63x
V4: OMP Threads (1T)	1.781	1.23x	10.129	1.22x	7171.75	1.04x
V5: Hybrid AVX2 (1T)	0.943	2.33x	11.224	1.10x	5494.46	1.36x
V6: Hybrid SIMD (1T)	2.229	0.98x	24.113	0.51x	11189.9	0.67x

Table 4: Single-thread performance. Speedup relative to V1 scalar baseline.

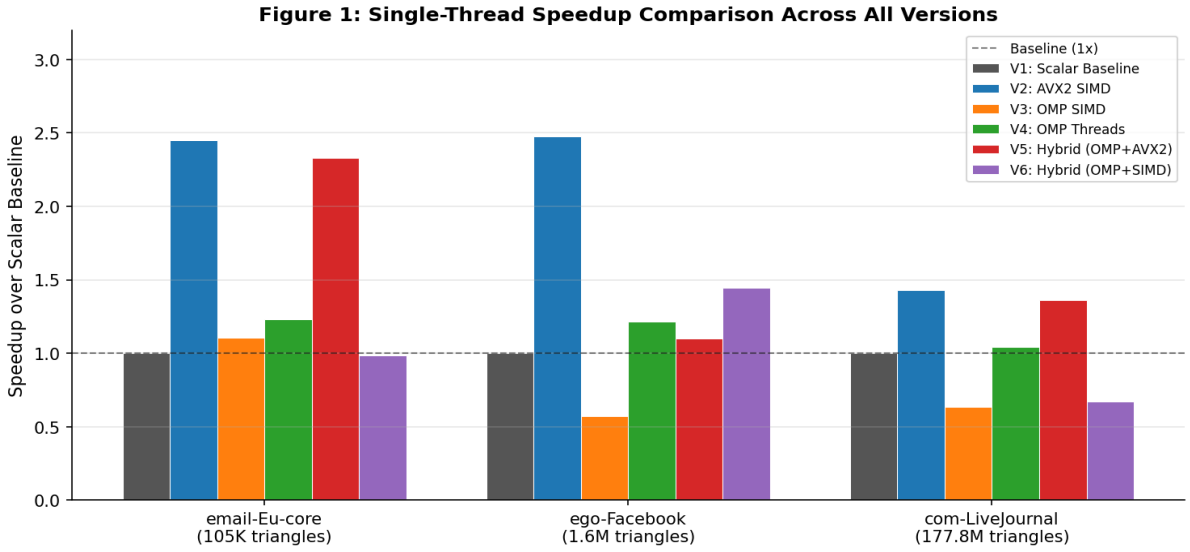

Figure 1: Single-thread speedup comparison across all versions and datasets.

6.2 Thread Scalability

Version	Dataset	1 Thread	2 Threads	4 Threads	8 Threads
V4: OMP Threads	D1	1.23x	2.43x	4.93x	7.27x
V4: OMP Threads	D2	1.22x	2.50x	3.97x	6.58x
V4: OMP Threads	D3	1.04x	1.81x	2.82x	4.39x
V5: Hybrid AVX2	D1	2.33x	4.34x	8.03x	13.78x
V5: Hybrid AVX2	D2	1.10x	1.58x	2.78x	5.73x
V5: Hybrid AVX2	D3	1.36x	2.51x	3.98x	5.42x

V6: Hybrid SIMD	D1	0.98x	1.71x	2.64x	4.96x
V6: Hybrid SIMD	D2	1.44x	2.56x	4.03x	6.05x
V6: Hybrid SIMD	D3	0.67x	1.25x	2.02x	2.84x

Table 5: Speedup at 1, 2, 4, and 8 threads relative to scalar baseline (V1).

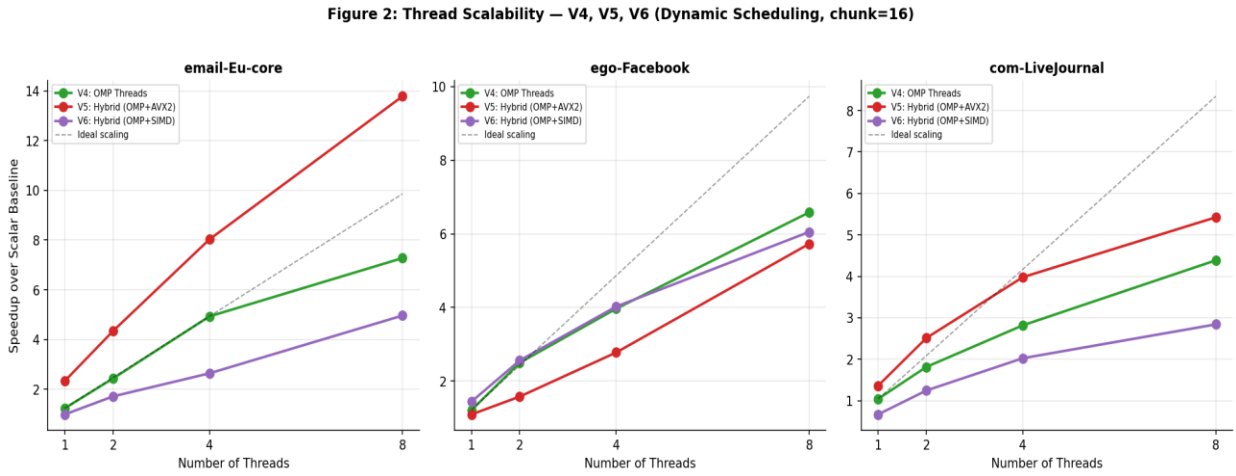

Figure 2: Thread scalability curves for V4, V5, V6 across all three datasets.

6.3 Best Configuration at 8 Threads

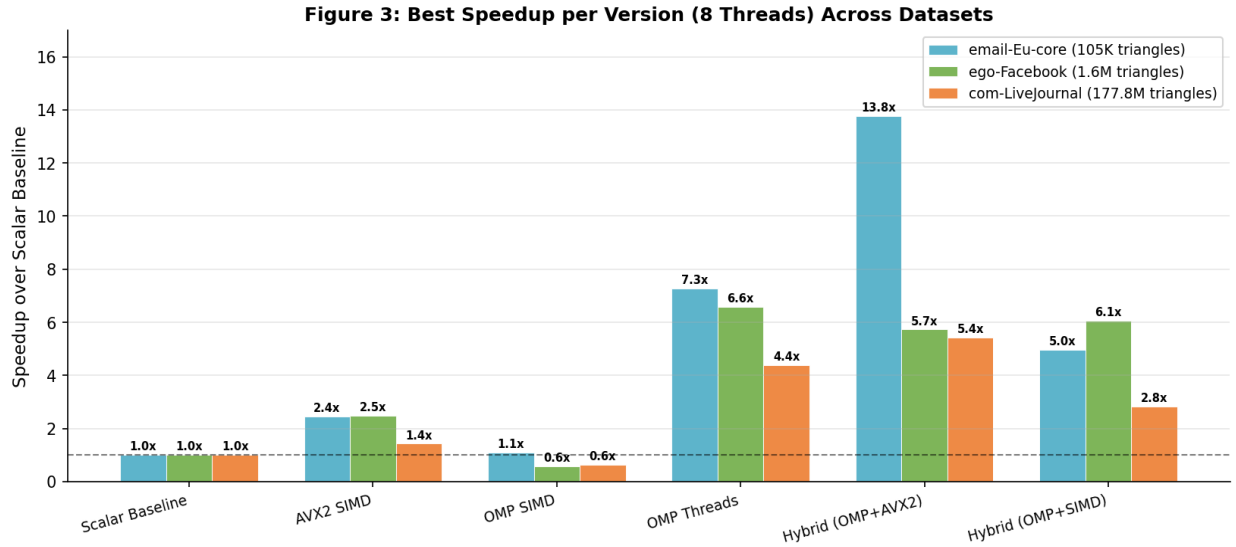

Figure 3: Speedup at 8 threads for all versions across all datasets.

6.4 Execution Time Comparison

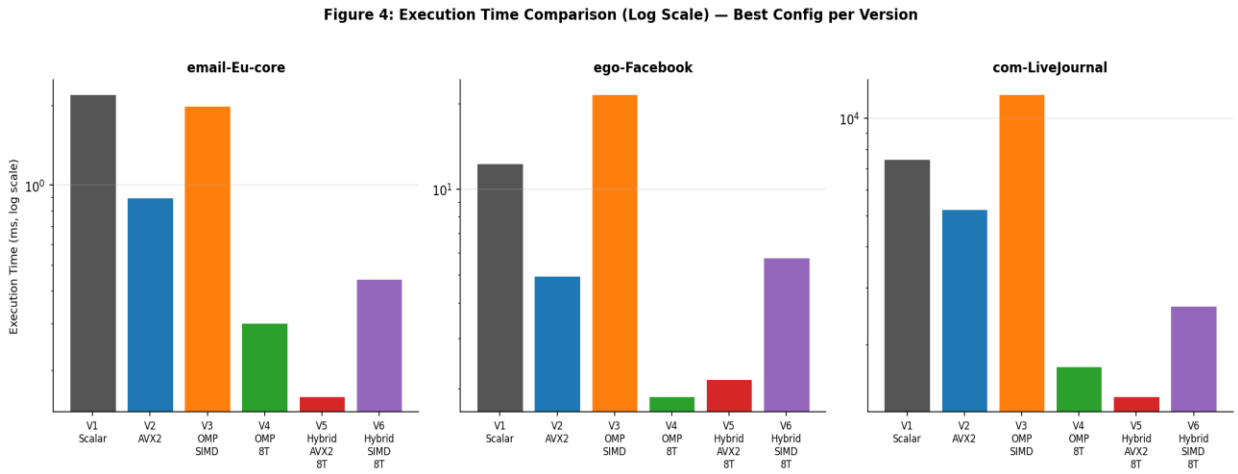

Figure 4: Absolute execution times (log scale) for best configuration per version.

7. Discussion and Analysis

7.1 Best Performing Variant

V5 (Hybrid OMP Threads + AVX2) is the best overall variant, achieving 13.78x speedup on D1 and 5.42x on D3 at 8 threads. This validates the design principle: combining two orthogonal parallelism sources (8 threads x 8-wide SIMD) yields greater speedup than either alone.

7.2 Why V2 Outperforms V3 (AVX2 vs OMP SIMD)

V2 (manual AVX2, 2.45-2.48x on D1/D2) consistently beats V3 (OMP SIMD, 0.57-1.10x). Two reasons: (1) V2 uses the O(n+m) merge algorithm which is optimal for similar-sized lists, while V3 restructures to O(n*m) for vectorizability. (2) Manual intrinsics allow precise control over register use and pointer advancement that the compiler cannot always replicate. The compiler may also fail to fully vectorize due to data dependencies it cannot prove away.

7.3 Why V4 Scales Well but Plateaus

V4 (OMP threads only) achieves 7.27x at 8 threads on D1, close to linear scaling. However on D3 (com-LiveJournal) it only achieves 4.39x. This is because D3 has longer adjacency lists — the per-intersection work is larger and memory bandwidth becomes a bottleneck. With 8 threads competing for the same memory bus, bandwidth saturation limits further scaling.

7.4 V3 Sub-Linear Speedup

V3 (OMP SIMD) performs worse than scalar on D2 (0.57x) and D3 (0.63x). The O(n*m) complexity directly explains this: on D2 and D3, adjacency lists are significantly longer after reordering than on D1. The SIMD benefit on the inner loop does not compensate for the increased number of comparisons. V3 is suitable only for very small graphs with short adjacency lists.

7.5 Memory Behavior

The degree-reordering optimization is critical for cache efficiency. Before reordering, a high-degree vertex (e.g., degree 800) would have a large upper-triangular adjacency list. After reordering, its list shrinks dramatically (max out-degree ~34 on email-Eu-core, down from 347 original max degree), improving L1/L2 cache hit rates during intersection. This is why preprocessing time is a worthwhile investment — it pays off across all subsequent intersection operations.

References

[1] A. S. Tom et al., "Exploring Optimizations on Shared-memory Platforms for Parallel Triangle Counting Algorithms," 2017.

[2] J. Zhang et al., "Preliminary Exploration of Large-Scale Triangle Counting on Shared-Memory Multicore System," IEEE HPEC, 2018.

[3] SNAP Datasets: https://snap.stanford.edu/data/